LICITRA-MMR: A Merkle Mountain Range Ledger Primitive for Cryptographic Runtime Accountability in Agentic AI Systems

Narendra Kumar Nutalapati
Independent Researcher, LICITRA Research Program
Portland, Oregon, USA
narendrakumar.nutalapati@gmail.com

Abstract

Agentic AI systems that make autonomous decisions in production create a new class of accountability requirement: the ability to reconstruct, after the fact, the exact decision chain with cryptographic verifiability — a stronger guarantee than operational logs alone provide. Existing logging approaches — including mutable database logs, hash chains, and general-purpose tamper-evident stores not tailored for agentic governance — provide observability but do not explicitly address independently verifiable runtime decision chains. We present LICITRA-MMR, an open-source ledger primitive that combines a Merkle Mountain Range (MMR) data structure with per-organization epoch anchoring, a versioned canonical JSON specification, and an atomic two-phase commit pipeline. At a block size of 1,000 events, LICITRA-MMR produces inclusion proofs requiring 14 SHA-256 operations, verifies a full epoch chain of 1,000 epochs in under 1 ms, and supports full leaf rehash of 1 million events in approximately 2 seconds. The system is fully open-source under MIT license with a tagged release, reproducible test suite, and evidence bundles. LICITRA-MMR is designed as a foundational cryptographic audit layer for agentic AI governance systems.

CCS Concepts

• **Security and privacy** → **Digital signatures**; **Tamper-proof and tamper-evident designs**; • **Computer systems organization** → *Dependable and fault-tolerant systems and networks.*

Keywords

Merkle Mountain Range, agentic AI, cryptographic audit, runtime integrity, tamper-evident logging, AI governance

1 Introduction

Regulatory frameworks for high-risk AI systems are converging on a common implicit technical requirement: that organizations can reconstruct AI decision chains after an incident with sufficient integrity to satisfy legal and audit scrutiny. The EU AI Act Article 12 [7] requires that high-risk AI systems generate logs that are traceable and sufficient for post-market monitoring. The Colorado AI Act [8] (SB 24-205, effective June 30, 2026) requires deployers to document AI decision-making processes and conduct impact assessments. Both frameworks are consistent with an audit capability that most production agentic AI stacks do not currently provide.

The gap is primarily technical rather than conceptual. Most teams deploy agentic AI with mutable database logs, cloud logging pipelines, or dashboard-based observability. These systems answer the question: *what does our log say happened?* They do not answer: *can we prove the log was not modified after the incident?* The distinction matters because administrators can rewrite database rows, cloud logs can be truncated or filtered, and cleanup scripts can alter historical records. A post-incident forensic investigation that relies entirely on operator-controlled logs provides no independent verification guarantee.

Agentic systems intensify this problem. Unlike single-inference ML systems, autonomous agents make multi-step decisions — each step potentially triggering real-world actions — without per-action human oversight. The decision chain that produced a harmful outcome may span dozens of sequential operations, each dependent on prior context. Reconstructing that chain requires not just that each event was logged, but that the sequence is complete, ordered, and unmodified.

We make the following contributions:

(1) **Canonical JSON Specification.** A versioned, cross-implementation deterministic serialization format (CANONICAL_JSON_SPEC v1.0) that ensures identical byte sequences from identical logical payloads across independent implementations — a prerequisite for deterministic verification.
(2) **MMR-Based Per-Organization Tamper-Evident Ledger.** A Merkle Mountain Range data structure providing $O(\log n)$ inclusion proofs and append-only semantics, with per-organization epoch anchoring that chains epoch roots into a hash-linked sequence resistant to retroactive modification.
(3) **Atomic Two-Phase Commit Pipeline.** A propose → policy → commit pipeline in which every MMR append, event record, and epoch finalization occurs within a single database transaction, preventing partial writes that could leave the ledger in an inconsistent state.
(4) **Open-Source Implementation with Reproducible Evidence.** A fully tested (11/11 passing), MIT-licensed implementation at tagged release v0.1.0-mvp, with reproducible JSON evidence bundles covering five tamper-detection scenarios.

2 Background

2.1 Merkle Mountain Ranges

A Merkle Mountain Range is an append-only authenticated data structure introduced by Peter Todd in the context of Bitcoin timestamping [1]. Unlike many Merkle tree implementations, in which insertion requires updating internal nodes along the rightmost path, an MMR avoids modifying previously committed subtrees. It maintains a forest of perfect binary trees (peaks) that are merged as new leaves are appended. This property makes MMRs ideal for append-only audit logs: each new element extends the structure in $O(\log n)$ time and space without modifying any existing node.

Formally, an MMR of size n stores $2n - \text{popcount}(n)$ nodes, where $\text{popcount}(n)$ is the number of set bits in the binary representation of n. The peaks of the MMR are the roots of the constituent perfect binary trees. A single MMR root is produced by bagging the peaks: hashing them right-to-left into a single digest. An inclusion proof for a leaf at position p consists of the sibling hashes on the path from p to its subtree peak, plus the remaining peaks, enabling logarithmic verification without access to the full node set.

At BLOCK_SIZE = 1,000 (our default epoch size), the MMR contains 1,994 nodes and produces inclusion proofs requiring exactly 14 hash operations. This compares favorably to a standard Merkle tree over the same leaf count, which requires $\lceil \log_2(1000) \rceil = 10$ hashes per proof path but requires updating internal nodes on insertion. The formal optimality of the MMR for append-only accumulators was established by Bonneau et al. [3].

2.2 Epoch Anchoring

LICITRA-MMR partitions the event stream into fixed-size epochs of BLOCK_SIZE events. When an epoch reaches capacity, an epoch record is finalized containing:

```
epoch_hash = SHA-256(
  prev_epoch_hash
  || mmr_root
  || SHA-256(metadata))
```

where `prev_epoch_hash` is the hash of the preceding epoch (or a 32-byte genesis value for epoch 0), `mmr_root` is the bagged peak hash of the epoch's MMR, and `metadata` is a canonical JSON document containing the organization ID, epoch ID, sequence range, event count, and ledger version.

This construction chains epochs into a hash-linked sequence analogous to a blockchain header chain, but without distributed consensus overhead. Modifying any event within an epoch invalidates that epoch's MMR root, which invalidates all subsequent epoch hashes. An auditor who holds the current epoch hash and any historical event can verify the complete chain of custody from genesis to present.

2.3 The Agentic AI Accountability Gap

Contemporary agentic AI systems — systems that autonomously plan and execute multi-step tasks using large language models as reasoning engines — introduce accountability challenges not present in traditional ML deployments. In a single-inference deployment, a harmful output is a discrete event that can be logged with a timestamp and input/output pair. In an agentic system, a harmful outcome may result from a sequence of individually benign decisions that collectively produce an unintended effect. Attributing the outcome requires reconstructing the full decision chain: which prompts the agent saw, which tools it invoked, which intermediate results it relied upon, and which final action it executed.

Existing logging infrastructure is not designed for this requirement. Application logs capture events as they occur but provide no tamper-evidence guarantees. Observability platforms aggregate telemetry but rely on operator-controlled infrastructure. Neither provides the forensic integrity consistent with emerging AI regulation: the ability for an external party to verify, independently of the operator, that a complete and unmodified record of the decision chain exists.

3 Threat Model

LICITRA-MMR is designed to protect the integrity of an append-only audit ledger against post-hoc modification. We consider the following attacker classes and explicitly state what the system does and does not protect against.

3.1 Protected Against

T1 — Insider / Database Administrator. An attacker with direct write access to the database attempts to modify the `canonical_json` field of a committed event. The leaf hash stored separately in the `leaf_hash` column will no longer match the recomputed SHA-256 of the modified canonical JSON, and MMR verification will fail at the affected leaf position.

T2 — Log Scrubber / Row Deletion. An attacker deletes one or more event rows to suppress evidence of a specific decision. The monotonic per-organization sequence number creates a detectable gap: verification will identify missing sequence positions, and the MMR node set will contain orphaned nodes at the deleted leaf positions.

T3 — Post-Incident Epoch Modification. An attacker modifies a historical epoch hash to hide evidence of prior tampering. Because each epoch hash is computed over the previous epoch hash, any modification to a historical epoch invalidates all subsequent epoch hashes.

T4 — Selective History Presentation. An operator presents only a subset of the audit log to an external auditor, claiming it is complete. An independent auditor can verify completeness by recomputing the MMR root from the presented events and checking it against the claimed epoch hash. Any omitted events will produce a root mismatch.

3.2 Not Protected Against

N1 — Compromised Application Layer. LICITRA-MMR commits whatever the application layer submits. Pre-execution integrity is the responsibility of LICITRA-SENTRY, discussed in Section 12.

N2 — Epoch Hash Without Signing. The current implementation computes epoch hashes but does not sign them with a private key. Ed25519 epoch signing is planned for v0.2 and discussed in Section 10.

N3 — Availability Attacks. LICITRA-MMR does not protect against denial-of-service attacks against the underlying database or API layer.

3.3 Trust Assumptions

The system assumes: (1) SHA-256 is collision-resistant; (2) the database storage layer preserves byte-level fidelity of stored values; (3) at least one honest party retains a copy of a recent epoch hash against which verification can be anchored. This assumption is structurally equivalent to Certificate Transparency's reliance on at least one honest log monitor [2]. No assumption is made about the trustworthiness of the operator beyond assumption (2).

4 Security Properties

We formalize the core integrity guarantees provided by LICITRA-MMR. These are not full proofs; they are precise statements of what the system guarantees under standard cryptographic assumptions.

4.1 Append-Only Integrity

Let $L = (e_1, e_2, \ldots, e_n)$ be the ordered sequence of committed events for an organization within an epoch. Define the epoch hash as:

$$\begin{aligned} H_{\text{epoch}} = \text{SHA-256}(&\texttt{prev_epoch_hash} \\ &\| \text{ MMR_root}(L) \\ &\| \text{ SHA-256}(\texttt{metadata})) \end{aligned}$$

Under the assumption that SHA-256 is collision-resistant, any adversary who modifies, deletes, reorders, or inserts an event in L after H_{epoch} has been externally anchored must produce either a second preimage for SHA-256 or a collision in the MMR construction. Therefore, retroactive modification of committed history without detection is computationally infeasible under standard hash function assumptions.

4.2 Inclusion Soundness

For any event e_i committed in epoch k, the inclusion proof π_i produced by the MMR satisfies:

$$\text{Verify}(e_i,\ \pi_i,\ \text{MMR_root}_k) = \texttt{true}$$

if and only if e_i was included in the committed set L_k at the time of epoch finalization. Any modification to e_i or to the node path represented in π_i requires either: (1) a collision in SHA-256, or (2) modification of the persisted node set — both of which are detectable under the append-only integrity model above.

4.3 Trust Model Limitation

These guarantees are relative. If an adversary controls both the database and all external epoch anchors, they can reconstruct a consistent but false history. LICITRA-MMR therefore provides tamper-evidence relative to at least one honest anchor holder, consistent with the trust assumption stated in Section 3.3. Epoch signing (planned for v0.2) will strengthen this guarantee by binding each epoch to a key holder independent of the database.

5 System Design

5.1 Canonical JSON Specification

A prerequisite for deterministic verification across independent implementations is that identical logical payloads produce identical byte sequences. LICITRA-MMR defines a versioned canonical JSON specification (CANONICAL_JSON_SPEC v1.0) with the rules shown in Table 1.

Known limitation. The specification does not currently normalize floating-point values. The integer `1` and the float `1.0` produce different byte sequences and therefore different leaf hashes. Callers must ensure consistent numeric types. Full float normalization per RFC 8785 §3.2.2 is planned for v1.1.

Table 1: Canonical JSON Specification v1.0

Property	Value
Character encoding	UTF-8, no BOM
Key ordering	Lexicographic by Unicode code point, recursive
Key-value separator	`:` (no spaces)
Item separator	`,` (no spaces)
Non-ASCII characters	Preserved as literal UTF-8
NaN / Infinity	Prohibited; raises `ValueError`
Whitespace	None outside string values

5.2 MMR Structure and Leaf Hashing

Each event payload is serialized to canonical JSON bytes and hashed:

$$\texttt{leaf_hash} = \text{SHA-256}(\text{canonical_bytes}(\text{payload}))$$

The leaf hash is appended to the epoch's MMR via `append_leaf()`, which adds the leaf node and merges sibling pairs upward. Each merge produces one parent node:

$$\texttt{parent_hash} = \text{SHA-256}(\texttt{left}\|\texttt{right})$$

All nodes are persisted to the `mmr_nodes` table in the same database transaction as the event record. The MMR root is never stored directly; it is recomputed from persisted nodes on demand, ensuring the root always reflects actual persisted state.

5.3 Epoch Chaining

Events are assigned to epochs by monotonic per-organization sequence number. When an epoch reaches BLOCK_SIZE events, `_finalize_epoch()` is invoked within the same transaction: (1) load all MMR nodes for the epoch; (2) compute MMR root by bagging peaks; (3) serialize epoch metadata to canonical JSON; (4) compute epoch hash; (5) persist the epoch record. The genesis epoch uses a fixed 32-byte zero string as `prev_epoch_hash`.

5.4 Two-Phase Commit Pipeline

Agent decisions enter the ledger through a two-phase pipeline:

Phase 1 — Propose. The agent submits a proposed action as a JSON document. The pipeline serializes it, evaluates it against the policy engine, and persists a `StagedEvent` record. If the proposal is rejected and `EMIT_BLOCKED_ACTION` is enabled, a `blocked_action` audit event is immediately committed to the MMR.

Phase 2 — Commit. An approved staged event is committed via `_insert_event()`, which: assigns a monotonic sequence number; serializes the payload; computes the leaf hash; appends to the MMR; persists all new MMR nodes; persists the event record; and finalizes the epoch if BLOCK_SIZE is reached. All operations occur within a single database transaction using a single `db.commit()` at the pipeline boundary.

5.5 Verification

An external auditor can verify any historical event independently: (1) *Leaf verification*: recompute the leaf hash and compare to the stored value; (2) *Inclusion proof*: verify the MMR proof path against

the epoch root; (3) *Epoch chain*: recompute each epoch hash from a trusted anchor and compare to stored values.

6 Implementation

LICITRA-MMR is implemented in Python 3.11 using FastAPI, SQLAlchemy, and PostgreSQL. The implementation is available at https://github.com/narendrakumarnutalapati/licitra-mmr-core under MIT license at tagged release v0.1.0-mvp.

6.1 Key Design Decisions

MMR recomputed from persisted nodes. The MMR root is never cached in application memory. Every MMR operation loads the ordered node list from the database, ensuring the root always reflects persisted state and any direct database modification is immediately detectable.

DEV_MODE endpoint gating. Tamper and reset endpoints return HTTP 403 unless `DEV_MODE=true` is explicitly set, with `false` as the default. The `.env` file is excluded from version control.

Atomic transaction boundary. The commit pipeline uses a single `db.commit()` at the pipeline boundary. All intermediate writes use `db.flush()` to assign database IDs without committing, ensuring any failure results in a full rollback.

SHA-256 via Python hashlib. All cryptographic operations use Python's `hashlib.sha256`, which wraps OpenSSL. SHA-256 is FIPS 180-4 compliant and accepted by all major compliance frameworks, making it preferable to BLAKE3 for compliance-oriented deployments despite BLAKE3's superior throughput.

6.2 Test Suite and Evidence Bundles

The implementation includes 11 passing tests covering event proposal and commitment, MMR node persistence, epoch finalization and hash chaining, tamper detection, row deletion detection, and inclusion proof verification. Five reproducible evidence scenarios are provided as JSON bundles: S01 (normal commit), S02 (leaf tamper), S03 (epoch tamper), S04 (row deletion), and S05 (cross-epoch verification).

7 Competitive Analysis

Table 2 compares LICITRA-MMR against four related systems.

immudb [4] is a general-purpose tamper-evident database using a Merkle tree over a linear log. It provides inclusion proofs and independent verification, making it the closest architectural peer. For use cases requiring only tamper-evident storage without agentic governance semantics, immudb is a mature alternative. LICITRA-MMR is distinguished by its governance architecture: epoch anchoring, canonical JSON specification, and integration with a pre-execution authorization layer.

Certificate Transparency logs [2] provide a globally auditable append-only log for TLS certificates with gossip-based consistency across independent operators. CT logs solve a different problem: proving certificate existence in a publicly auditable multi-operator log. They require external operators, have no per-organization namespacing, and are not designed for high-frequency per-org event streams.

Oktsec [5] provides agent identity verification and message integrity using Ed25519 signatures and SHA-256 hash chains, covering multiple OWASP Agentic Top 10 threat categories. LICITRA-MMR addresses a complementary layer: epoch-anchored, independently verifiable commitment of authorized agent decisions, rather than message-level integrity. The key architectural distinction is that Oktsec's hash chain cannot produce inclusion proofs without replaying the full chain; LICITRA's MMR enables $O(\log n)$ verification of any single event.

8 Theoretical Performance Evaluation

All estimates assume Python 3.11 `hashlib.sha256` on an Intel x86-64 processor (3.2 GHz, 32 GB RAM), with SHA-256 latency of approximately 500 ns per call as measured via `timeit` microbenchmark. Typical agent event payloads are assumed to be approximately 200 bytes. Database I/O latency dominates the commit path; cryptographic overhead is negligible in all scenarios.

8.1 Inclusion Proof Path Length

Table 3 shows proof path lengths at various epoch sizes. At the default block size of 1,000, an inclusion proof requires 14 SHA-256 operations and occupies 462 bytes in binary encoding.

8.2 Verification Cost at 1 Million Events

With block size 1,000, one million events produce 1,000 epochs. *Epoch chain verification* requires 2 SHA-256 operations per epoch (2,000 total), completing in approximately 1 ms. *Per-event inclusion proof* requires 14 SHA-256 operations and completes in approximately 7 μs per event. *Full leaf rehash* of all 1 million events requires approximately 2 seconds at 2 μs per event. *Sample verification* of 100 randomly selected events requires approximately 0.9 ms.

8.3 Commit Overhead

The amortized cryptographic overhead per commit is approximately 6 SHA-256 operations (≈ 3 μs). The dominant commit cost is database I/O: three write operations per commit.

8.4 Storage at Scale

At 1 million events with block size 1,000, the estimated storage footprint is approximately 564 MB: MMR nodes ≈217 MB (1,994,000 node rows), event rows ≈347 MB (1,000,000 rows at ≈364 bytes each), epoch rows < 1 MB. Storage scales linearly with event count. The MMR node overhead is approximately 2× the leaf count, consistent with the asymptotic minimum for append-only binary hash tree constructions.

9 Architectural Non-Goals

The following properties are explicitly outside the scope of LICITRA-MMR.

Not a consensus system. There is one writer and one ledger per organization. Consistency is enforced by database transaction semantics, not a consensus protocol.

Not a blockchain. The epoch chain is structurally similar to a blockchain header chain but is not a blockchain. There is no proof of work, proof of stake, token, peer-to-peer network, or distributed ledger.

Table 2: Comparison of tamper-evident audit systems for agentic AI

Feature	LICITRA-MMR	immudb	CT Logs	Oktsec
Data structure	MMR	Merkle tree	Merkle tree	Hash chain
Inclusion proof	$O(\log n)$	$O(\log n)$	$O(\log n)$	None
Epoch anchoring	✓	×	×	×
Per-org ledger	✓	Partial	×	✓
Pre-execution gate	× (SENTRY)	×	×	×
Explicit AI audit design	✓	×	×	Partial
Canonical JSON spec	✓versioned	×	×	×
Independent verification	✓	✓	✓	×
License	MIT	Apache-2.0	N/A	GPL-3.0
Regulatory framing	EU AI Act, CO AI Act	General	PKI/TLS	None

Table 3: MMR inclusion proof path length by epoch size

Epoch size (events)	Proof path (hashes)	MMR nodes	Proof size (binary)
100	8	197	264 B
500	13	994	429 B
1,000	14	1,994	462 B
2,000	15	3,994	495 B
10,000	17	19,995	561 B

Not Byzantine fault tolerant. The system assumes the database storage layer faithfully preserves byte-level fidelity of stored values. It does not protect against a Byzantine storage layer.

Not preventing application-layer falsification. LICITRA-MMR commits whatever the application layer submits. Pre-execution integrity is the scope of LICITRA-SENTRY, not LICITRA-MMR.

Not availability-protecting. Availability is the responsibility of the deployment infrastructure.

10 Limitations and Future Work

10.1 Float Normalization Gap

The canonical JSON specification does not normalize floating-point values. The integer `1` and the float `1.0` produce different byte sequences and different leaf hashes. For the primary use case of agent decision logging, this is unlikely to cause verification failures in practice. Full float normalization per RFC 8785 §3.2.2 is planned for CANONICAL_JSON_SPEC v1.1.

10.2 Unsigned Epoch Roots

The current implementation computes epoch hashes but does not sign them with a private key. An attacker who controls both the database and the application server can in principle reconstruct a consistent but false history. Ed25519 epoch signing is planned for v0.2. The epoch record schema reserves a `signature` field for this purpose.

10.3 Single-Operator Trust Model

LICITRA-MMR is designed for single-operator deployments. Multi-party witnessing — where multiple independent operators each hold a copy of the current epoch hash — is a direction for future work, analogous to the gossip protocol in Certificate Transparency [2].

10.4 Pre-Execution Integrity

LICITRA-MMR records what was committed. It does not guarantee that what was committed accurately reflects what the agent was authorized to do. Pre-execution integrity is the scope of LICITRA-SENTRY.

11 Related Work

Merkle Mountain Ranges. MMRs were introduced by Peter Todd [1] and have been adopted in blockchain systems including Grin and Beam. The formal optimality of the MMR for append-only accumulators was established by Bonneau et al. [3]. LICITRA-MMR applies the MMR structure to per-organization agentic AI audit logs, a context not previously addressed in the literature.

Certificate Transparency. Laurie et al. [2] introduced CT logs for publicly auditing TLS certificate issuance. LICITRA-MMR shares the append-only authenticated structure but differs in trust model, scope, and operational requirements.

immudb. Codenotary's immudb [4] provides a tamper-evident database using a Merkle tree over an append-only log. It is the closest architectural peer; distinctions are described in Section 6.

Oktsec. The Oktsec framework [5] provides agent identity verification and message integrity. LICITRA-MMR addresses a complementary layer at the authorization and audit level.

NIST AI Risk Management Framework. The NIST AI RMF [6] provides governance guidance including auditability requirements. LICITRA-MMR provides a technical implementation of that requirement.

12 Conclusion

We have presented LICITRA-MMR, an open-source cryptographic ledger primitive for agentic AI runtime accountability. The system combines a Merkle Mountain Range with per-organization epoch anchoring, a versioned canonical JSON specification, and an atomic

two-phase commit pipeline to provide forensic integrity guarantees that mutable logging infrastructure cannot offer.

The three core contributions — the canonical specification, the MMR-based epoch-anchored ledger, and the atomic commit pipeline — address the gap between what emerging AI regulation implies (post-incident reconstructability with sufficient integrity for audit) and what most production agentic AI stacks currently provide (operator-controlled mutable logs).

LICITRA-MMR is the foundational layer of the broader LICITRA cryptographic governance stack. The next layer, LICITRA-SENTRY, adds a zero-trust control plane with pre-execution semantic contracts, agent identity verification, content inspection, and Chain of Intent authorization.

12.1 Deployment Considerations

Organizations deploying LICITRA-MMR should consider: (1) *Epoch hash anchoring* — store at least one epoch hash off-site via a notarization service, timestamping service, or publication to a public ledger, to prevent an operator from reconstructing a consistent false history; (2) *Key management preparation* — establish KMS or HSM infrastructure in advance of the v0.2 signing upgrade; (3) *Backup and retention* — the MMR node table grows at approximately 2× the event count; (4) *Regulatory export* — use the JSON evidence bundle format from the `licitra-mmr-evidence` repository for regulatory submissions.

Reproducibility. All code, test suites, and evidence bundles are available at https://github.com/narendrakumarnutalapati/licitra-mmr-core. The tagged release v0.1.0-mvp and all evidence bundles are permanently archived at https://doi.org/10.5281/zenodo.18796529.

References

[1] P. Todd. Merkle Mountain Ranges. OpenTimestamps, 2013. https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md

[2] B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. RFC 6962, Internet Engineering Task Force, 2013. https://www.rfc-editor.org/rfc/rfc6962

[3] J. Bonneau, J. Chen, M. Christ, and I. Karantaidou. Merkle Mountain Ranges are Optimal: On Witness Update Frequency for Cryptographic Accumulators. In *Advances in Cryptology – CRYPTO 2025*, LNCS vol. 16001, Springer, 2025. https://eprint.iacr.org/2025/234

[4] Codenotary. immudb: Immutable Database. https://immudb.io

[5] Oktsec. Agent Security Framework (v0.1). https://github.com/oktsec/oktsec Accessed: February 2026.

[6] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). https://doi.org/10.6028/NIST.AI.100-1, 2023.

[7] European Parliament. Regulation (EU) 2024/1689 on Artificial Intelligence (EU AI Act). Official Journal of the European Union, 2024.

[8] Colorado General Assembly. SB 24-205: Colorado Artificial Intelligence Act. Effective June 30, 2026. https://leg.colorado.gov/bills/sb24-205